

Cuaderno de ejercicios – APIs (3.0)

DAM | Programación de servicios y procesos

1. API – Hospital MedCare

Se solicita crear una API que responda a peticiones POST y GET para gestionar las citas médicas del Hospital MedCare. Un usuario (paciente) debe poder registrarse, iniciar sesión, consultar los médicos disponibles, pedir una cita, cancelar una cita y cerrar sesión.

Ficheros de datos:

- Pacientes.dat – datos de pacientes, formato (usuario:contraseña:nombre:dni).
- Pacientes_activos.dat – pacientes con sesión activa.
- Medicos.dat – listado de médicos, formato (idMedico:nombre:especialidad:horario:citasMax:citasActuales).
- Citas.dat – citas registradas, formato (idCita:idMedico:usuario:fecha).

Peticiones POST

Petición POST	Respuesta
<code>http://localhost:8080/medcare/register</code> Body: { "user": "nameUser", "pass": "passUser", "nombre": "Nombre Completo", "dni": "12345678A" }	El servidor comprueba si el usuario ya existe en "Pacientes.dat". Si no existe: • Añade la línea "nameUser:passUser:Nombre:dni". • Devuelve código HTTP 204 sin contenido. Si ya existe: • Devuelve código HTTP 409 con el mensaje "El nombre de usuario ya existe".
<code>http://localhost:8080/medcare/login</code> Body: { "user": "nameUser", "pass": "passUser" }	El servidor comprueba usuario y contraseña en "Pacientes.dat". Si está autorizado: • Añade el usuario a "Pacientes_activos.dat". • Devuelve código HTTP 204 sin contenido. Si no está autorizado: • Devuelve código 401 sin contenido.
<code>http://localhost:8080/medcare/cita</code> Body: { "user": "nameUser", "idMedico": "idMedico", "fecha": "dd/mm/yyyy" }	El servidor comprueba si el usuario está activo. Si está activo: • Comprueba si <code>citasActuales < citasMax</code> en "Medicos.dat". • Si hay hueco: - Incrementa <code>citasActuales</code> en 1 en "Medicos.dat". - Genera un <code>idCita</code> único (p.ej. <code>autoincremental</code>). - Añade la línea en "Citas.dat". - Devuelve código 204 sin contenido. • Si no hay hueco o el médico no existe: - Devuelve código 404 sin contenido. Si no está activo: • Devuelve código 401 sin contenido.

http://localhost:8080/medcare/cancelar Body: <pre>{ "user": "nameUser", "idCita": "idCita" }</pre>	El servidor comprueba si el usuario está activo. Si está activo: • Busca en "Citas.dat" la cita con idCita y user. • Si existe: - Elimina la línea de "Citas.dat". - Decrementa citasActuales en 1 en "Medicos.dat". - Devuelve código 204 sin contenido. • Si no existe: - Devuelve código 404 sin contenido. Si no está activo: • Devuelve código 401 sin contenido.
http://localhost:8080/medcare/logout Body: <pre>{ "user": "nameUser" }</pre>	El servidor comprueba si el usuario está activo. Si está activo: • Elimina el usuario de "Pacientes_activos.dat". • Devuelve código HTTP 204 sin contenido. Si no está activo: • Devuelve código 401 sin contenido.

Peticiones GET

Petición GET	Respuesta
http://localhost:8080/medcare/medicos ?user=nameUser	El servidor comprueba si el usuario está activo. Si está activo: • Devuelve código 200 con JSON: <pre>{ "medicos": [{ "idMedico": "1", "nombre": "Dr. García", "especialidad": "Cardiología", "horario": "Lunes 09:00", "citasLibres": 3 }] }</pre> Si no está activo: • Devuelve código 401 sin contenido.
http://localhost:8080/medcare/misCitas ?user=nameUser	El servidor comprueba si el usuario está activo. Si está activo: • Busca en "Citas.dat" las citas del usuario. • Si tiene citas: - Devuelve código 200 con JSON: <pre>{ "citas": [{ "idCita": "1", "medico": "Dr. García", "especialidad": "Cardiología", "fecha": "dd/mm/yyyy" }] }</pre> • Si no tiene citas: - Devuelve código 404 sin contenido. Si no está activo: • Devuelve código 401 sin contenido.

2. API – Tienda Online ShopEasy

Se solicita crear una API para gestionar una tienda online. Un usuario debe poder registrarse, iniciar sesión, cambiar su contraseña, consultar el catálogo de productos, añadir productos a su carrito, realizar un pedido, consultar sus pedidos y cerrar sesión.

Ficheros de datos:

- Usuarios.dat – formato (usuario:contraseña:email:direccion).
- Usuarios_activos.dat – usuarios con sesión activa.
- Productos.dat – formato (idProducto:nombre:precio:stock).
- Carritos.dat – formato (usuario:idProducto:cantidad).
- Pedidos.dat – formato (idPedido:usuario:idProducto:cantidad:fecha:total).

Peticiones POST

Petición POST	Respuesta
<code>http://localhost:8080/shopeasy/register</code> Body: { "user": "nameUser", "pass": "passUser", "email": "mail@mail.com", "direccion": "Calle X" }	Comprueba si el usuario ya existe en "Usuarios.dat". Si no existe: • Añade "user:pass:email:direccion" a "Usuarios.dat". • Devuelve código 204 sin contenido. Si ya existe: • Devuelve código 409 con mensaje "El nombre de usuario ya existe".
<code>http://localhost:8080/shopeasy/login</code> Body: { "user": "nameUser", "pass": "passUser" }	Comprueba usuario y contraseña en "Usuarios.dat". Si autorizado: • Añade usuario a "Usuarios_activos.dat". • Devuelve código 204. Si no autorizado: • Devuelve código 401 sin contenido.
<code>http://localhost:8080/shopeasy/changePwd</code> Body: { "user": "nameUser", "pass": "newPass" }	Comprueba si el usuario está activo. Si está activo: • Actualiza contraseña en "Usuarios.dat". • Devuelve código 204 sin contenido. Si no está activo: • Devuelve código 401 sin contenido.
<code>http://localhost:8080/shopeasy/carrito</code> Body: { "user": "nameUser", "idProducto": "idProducto", "cantidad": cantidadNumerica }	Comprueba si el usuario está activo. Si está activo: • Verifica que el producto existe y que cantidad <= stock en "Productos.dat". • Si hay stock suficiente: - Añade/actualiza la línea en "Carritos.dat". - Devuelve código 204 sin contenido. • Si no hay stock o producto no existe: - Devuelve código 404 sin contenido. Si no está activo: • Devuelve código 401 sin contenido.

	SUGERENCIA: Si el producto ya está en el carrito, actualiza la cantidad en lugar de duplicar la línea.
http://localhost:8080/shopeasy/pedido Body: <pre>{ "user": "nameUser" }</pre>	Comprueba si el usuario está activo. Si está activo: • Lee las líneas del usuario en "Carritos.dat". • Si el carrito no está vacío: - Descuenta el stock de cada producto en "Productos.dat". - Crea líneas en "Pedidos.dat" con fecha actual y precio total (precio × cantidad). - Elimina las líneas del usuario en "Carritos.dat". • Devuelve código 204 sin contenido. • Si el carrito está vacío: - Devuelve código 404 sin contenido. Si no está activo: • Devuelve código 401 sin contenido.
http://localhost:8080/shopeasy/logout Body: <pre>{ "user": "nameUser" }</pre>	Comprueba si el usuario está activo. Si está activo: • Elimina usuario de "Usuarios_activos.dat". • Devuelve código 204 sin contenido. Si no está activo: • Devuelve código 401 sin contenido.

Peticiones GET

Petición GET	Respuesta
http://localhost:8080/shopeasy/catalogo?user=nameUser	Comprueba si el usuario está activo. Si está activo: • Devuelve código 200 con JSON: <pre>{ "productos": [{ "idProducto": "1", "nombre": "Auriculares BT", "precio": 49.99, "stock": 20 }] }</pre> Si no está activo: • Devuelve código 401 sin contenido.
http://localhost:8080/shopeasy/misPedidos?user=nameUser	Comprueba si el usuario está activo. Si está activo: • Busca en "Pedidos.dat" las líneas del usuario. • Si tiene pedidos: - Devuelve código 200 con JSON: <pre>{ "pedidos": [{ "idPedido": "1", "producto": "Auriculares BT", "cantidad": 2, "total": 99.98, "fecha": "dd/mm/yyyy" }] }</pre> • Si no tiene pedidos:

	- Devuelve código 404 sin contenido. Si no está activo: • Devuelve código 401 sin contenido.
http://localhost:8080/shopeasy/ miCarrito ?user=nameUser	Comprueba si el usuario está activo. Si está activo: • Lee las líneas del usuario en "Carritos.dat". • Si tiene productos en el carrito: - Devuelve código 200 con JSON: <pre>{ "carrito": [{ "idProducto": "1", "nombre": "Auriculares BT", "cantidad": 2, "subtotal": 99.98 }] }</pre> • Si está vacío: - Devuelve código 404 sin contenido. Si no está activo: • Devuelve código 401 sin contenido.

3. API – Parking Inteligente SmartPark

Se solicita crear una API para gestionar el parking SmartPark. Un usuario debe poder registrarse, iniciar sesión, consultar las plazas disponibles, aparcar (reservar una plaza), liberar una plaza, consultar su historial y cerrar sesión. Además, debe poder registrar vehículos a su cuenta.

Ficheros de datos:

- Usuarios.dat – formato (usuario:contraseña:nombre).
- Usuarios_activos.dat – usuarios con sesión activa.
- Vehiculos.dat – formato (matricula:usuario:tipo), donde tipo es "coche", "moto" o "furgoneta".
- Plazas.dat – formato (idPlaza:tipo:disponible), donde disponible es "S" o "N".
- Ocupaciones.dat – formato (idPlaza:matricula:usuario:horaEntrada).
- Historial.dat – formato (idPlaza:matricula:usuario:horaEntrada:horaSalida).

Peticiones POST

Petición POST	Respuesta
http://localhost:8080/smartpark/register Body: <pre>{ "user": "nameUser", "pass": "passUser", "nombre": "Nombre Completo" }</pre>	Comprueba si el usuario ya existe en "Usuarios.dat". Si no existe: • Añade "user:pass:nombre" a "Usuarios.dat". • Devuelve código 204 sin contenido. Si ya existe: • Devuelve código 409 con mensaje "El nombre de usuario ya existe".
http://localhost:8080/smartpark/login Body: <pre>{ "user": "nameUser",</pre>	Comprueba usuario y contraseña en "Usuarios.dat". Si autorizado: • Añade usuario a "Usuarios_activos.dat". • Devuelve código 204.

<pre>"pass": "passUser" }</pre>	Si no autorizado: • Devuelve código 401 sin contenido.
<pre>http://localhost:8080/smartpark/ vehiculo Body: { "user": "nameUser", "matricula": "1234ABC", "tipo": "coche" }</pre>	Comprueba si el usuario está activo. Si está activo: • Comprueba que la matrícula no existe ya en "Vehiculos.dat". • Si no existe: - Añade "matricula:user:tipo" a "Vehiculos.dat". - Devuelve código 204 sin contenido. • Si ya existe: - Devuelve código 409 con mensaje "La matrícula ya está registrada". Si no está activo: • Devuelve código 401 sin contenido.
<pre>http://localhost:8080/smartpark/ aparcar Body: { "user": "nameUser", "matricula": "1234ABC" }</pre>	Comprueba si el usuario está activo. Si está activo: • Verifica que la matrícula pertenece al usuario en "Vehiculos.dat" y obtiene el tipo de vehículo. • Busca una plaza disponible ("S") del mismo tipo en "Plazas.dat". • Si hay plaza disponible: - Cambia "S" por "N" en "Plazas.dat". - Añade línea en "Ocupaciones.dat" con hora actual. - Devuelve código 204 sin contenido. • Si no hay plaza: - Devuelve código 404 sin contenido. Si no está activo: • Devuelve código 401 sin contenido.
<pre>http://localhost:8080/smartpark/ salir Body: { "user": "nameUser", "matricula": "1234ABC" }</pre>	Comprueba si el usuario está activo. Si está activo: • Busca en "Ocupaciones.dat" la matrícula del usuario. • Si existe la ocupación: - Elimina la línea de "Ocupaciones.dat". - Cambia "N" por "S" en "Plazas.dat". - Añade la línea en "Historial.dat" con hora salida. - Devuelve código 204 sin contenido. • Si no existe: - Devuelve código 404 sin contenido. Si no está activo: • Devuelve código 401 sin contenido.
<pre>http://localhost:8080/smartpark/ logout Body: { "user": "nameUser" }</pre>	Comprueba si el usuario está activo. Si está activo: • Elimina usuario de "Usuarios_activos.dat". • Devuelve código 204 sin contenido. Si no está activo: • Devuelve código 401 sin contenido.

Peticiones GET

Petición GET	Respuesta
<pre>http://localhost:8080/smartpark/plazas ?user=nameUser&tipo=coche</pre>	Comprueba si el usuario está activo. Si está activo: - Filtra "Plazas.dat" por tipo (si se indica). - Devuelve código 200 con JSON: <pre>{ "plazas": [{ "idPlaza": "A1", "tipo": "coche", "disponible": "S" }] }</pre> Si no está activo: - Devuelve código 401 sin contenido.
<pre>http://localhost:8080/smartpark/historial ?user=nameUser</pre>	Comprueba si el usuario está activo. Si está activo: - Busca en "Historial.dat" las líneas del usuario. - Si tiene historial: - Devuelve código 200 con JSON: <pre>{ "historial": [{ "idPlaza": "A1", "matricula": "1234ABC", "entrada": "hh:mm dd/mm/yyyy", "salida": "hh:mm dd/mm/yyyy" }] }</pre> - Si no tiene historial: - Devuelve código 404 sin contenido. Si no está activo: - Devuelve código 401 sin contenido.

4. API con Token – Academia de Idiomas LinguaAPI ★ Avanzado

Se solicita crear una API con autenticación por token para gestionar los cursos de la academia LinguaAPI. Tras hacer login, la API devuelve un token único que deberá incluirse en todas las peticiones. El token se elimina al hacer logout.

Ficheros de datos:

- Profesores.dat – formato (usuario:contraseña).
- Cursos.dat – formato (idCurso:nombre:idioma:nivel:plazasTotal:plazasLibres).
- Alumnos.dat – formato (idAlumno:nombre:email:idCurso).

Peticiones POST

Petición POST	Respuesta
<pre>http://localhost:8080/lingua/login</pre> Body: <pre>{ "user": "nameUser", "pass": "passUser" }</pre>	Comprueba si el usuario existe en "Profesores.dat". Si está autorizado: - Genera un token UUID truncado. - Guarda el token en una lista (ArrayList). - Devuelve código 200 con el token. Si no está autorizado: - Devuelve código 401 sin contenido.

	<pre>* String uuid = UUID.randomUUID().toString(); String token = uuid.split("-")[0];</pre>
http://localhost:8080/lingua/nuevoCurso Body: <pre>{ "token": "tokenUser", "nombre": "Inglés B2", "idioma": "Inglés", "nivel": "B2", "plazas": plazasNumericas }</pre>	Comprueba si el token está en la lista. Si está autorizado: • Genera un idCurso autoincremental. • Añade la línea en "Cursos.dat" con plazasLibres = plazas. • Devuelve código 204 sin contenido. Si no está autorizado: • Devuelve código 401 sin contenido.
http://localhost:8080/lingua/matricular Body: <pre>{ "token": "tokenUser", "nombre": "Nombre Alumno", "email": "email@mail.com", "idCurso": "idCurso" }</pre>	Comprueba si el token está en la lista. Si está autorizado: • Comprueba plazasLibres > 0 en "Cursos.dat". • Si hay plazas: - Genera idAlumno autoincremental. - Añade línea en "Alumnos.dat". - Decrementa plazasLibres en 1 en "Cursos.dat". • Devuelve código 204 sin contenido. • Si no hay plazas o el curso no existe: - Devuelve código 404 sin contenido. Si no está autorizado: • Devuelve código 401 sin contenido.
http://localhost:8080/lingua/logout Body: <pre>{ "token": "tokenUser" }</pre>	Comprueba si el token está en la lista. Si está autorizado: • Elimina el token de la lista. • Devuelve código 204 sin contenido. Si no está autorizado: • Devuelve código 401 sin contenido.

Peticiones GET

Petición GET	Respuesta
http://localhost:8080/lingua/cursos?token=tokenUser	Comprueba si el token está en la lista. Si está autorizado: • Devuelve código 200 con JSON: <pre>{ "cursos": [{ "idCurso": "1", "nombre": "Inglés B2", "idioma": "Inglés", "nivel": "B2", "plazasLibres": 8 }] }</pre> Si no está autorizado: • Devuelve código 401 sin contenido.
http://localhost:8080/lingua/alumnos?token=tokenUser&idCurso=idCurso	Comprueba si el token está en la lista. Si está autorizado: • Filtra "Alumnos.dat" por idCurso. • Si hay alumnos: - Devuelve código 200 con JSON: <pre>{ "alumnos": [{ "idAlumno": "1",</pre>

```
"nombre": "Ana López",  
"email": "ana@mail.com" }
```

```
] }
```

- Si no hay alumnos:
 - Devuelve código 404 sin contenido.

Si no está autorizado:

- Devuelve código 401 sin contenido.